Recursion Backtracking Study Guide

1. Understanding Backtracking

What is Backtracking?

Backtracking is a problem-solving technique that uses recursion to explore all possible solutions by trying partial solutions and abandoning them ("backtracking") if they cannot lead to a complete solution. It's like solving a maze - you try a path, and if it leads to a dead end, you go back and try a different path.

Key Characteristics of Backtracking

1. **Recursive Nature**: Uses recursion to explore solutions
2. **Trial and Error**: Tries different possibilities systematically
3. **Undo Operation**: Backs out of choices that don't work
4. **Complete Search**: Explores all possible solutions if needed

Basic Backtracking Template

```java
public boolean backtrack(parameters) {
    // Base case - check if solution is complete
    if (isComplete()) {
        return true; // or process the solution
    }

    // Try all possible choices
    for (each possible choice) {
        // Make the choice
        makeChoice();

        // Recursively explore
        if (backtrack(updatedParameters)) {
            return true; // Solution found
        }

        // Undo the choice (backtrack)
        undoChoice();
    }

    return false; // No solution found
}
```

When to Use Backtracking

- Finding all possible solutions to a problem
- Optimization problems where you need the best solution
- Constraint satisfaction problems
- Puzzles and games (N-Queens, Sudoku, etc.)
- Pathfinding in mazes or graphs

Backtracking vs Regular Recursion

Regular Recursion: Breaks down a problem into smaller subproblems **Backtracking**: Explores multiple paths and backtracks when a path doesn't work

2. Practice Questions

Example 1: N-Queens Problem

The N-Queens problem asks to place N queens on an N×N chessboard so that no two queens attack each other.

java

```java
public class NQueens {
    private int n;
    private int[] board; // board[i] = column of queen in row i

    public boolean solveNQueens(int n) {
        this.n = n;
        this.board = new int[n];
        return backtrack(0);
    }

    private boolean backtrack(int row) {
        // Base case: all queens placed
        if (row == n) {
            printSolution();
            return true;
        }

        // Try placing queen in each column of current row
        for (int col = 0; col < n; col++) {
            if (isSafe(row, col)) {
                // Make choice
                board[row] = col;

                // Recurse
                if (backtrack(row + 1)) {
                    return true;
                }

                // Backtrack (undo choice)
                board[row] = -1;
            }
        }

        return false;
    }

    private boolean isSafe(int row, int col) {
        for (int i = 0; i < row; i++) {
            // Check column and diagonals
            if (board[i] == col ||
                Math.abs(board[i] - col) == Math.abs(i - row)) {
                return false;
            }
```

```java
        }
        return true;
    }

    private void printSolution() {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(board[i] == j ? "Q " : ". ");
            }
            System.out.println();
        }
    }
}
```

Example 2: Generate All Permutations

Generate all possible permutations of a given array.

```java
import java.util.*;

public class Permutations {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        List<Integer> current = new ArrayList<>();
        boolean[] used = new boolean[nums.length];

        backtrack(nums, current, used, result);
        return result;
    }

    private void backtrack(int[] nums, List<Integer> current,
                           boolean[] used, List<List<Integer>> result) {
        // Base case: permutation is complete
        if (current.size() == nums.length) {
            result.add(new ArrayList<>(current));
            return;
        }

        // Try each unused number
        for (int i = 0; i < nums.length; i++) {
            if (!used[i]) {
                // Make choice
                current.add(nums[i]);
                used[i] = true;

                // Recurse
                backtrack(nums, current, used, result);

                // Backtrack
                current.remove(current.size() - 1);
                used[i] = false;
            }
        }
    }
}
```

Example 3: Subset Generation

Generate all possible subsets of a given set.

```java
import java.util.*;

public class Subsets {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        List<Integer> current = new ArrayList<>();

        backtrack(nums, 0, current, result);
        return result;
    }

    private void backtrack(int[] nums, int start,
                           List<Integer> current, List<List<Integer>> result) {
        // Add current subset to result
        result.add(new ArrayList<>(current));

        // Try including each remaining element
        for (int i = start; i < nums.length; i++) {
            // Make choice
            current.add(nums[i]);

            // Recurse with next index
            backtrack(nums, i + 1, current, result);

            // Backtrack
            current.remove(current.size() - 1);
        }
    }
}
```

Example 4: Sudoku Solver

Solve a 9×9 Sudoku puzzle using backtracking.

java

```java
public class SudokuSolver {
    public boolean solveSudoku(char[][] board) {
        return backtrack(board);
    }

    private boolean backtrack(char[][] board) {
        // Find empty cell
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == '.') {
                    // Try digits 1-9
                    for (char digit = '1'; digit <= '9'; digit++) {
                        if (isValid(board, row, col, digit)) {
                            // Make choice
                            board[row][col] = digit;

                            // Recurse
                            if (backtrack(board)) {
                                return true;
                            }

                            // Backtrack
                            board[row][col] = '.';
                        }
                    }
                    return false; // No valid digit found
                }
            }
        }
        return true; // All cells filled
    }

    private boolean isValid(char[][] board, int row, int col, char digit) {
        // Check row, column, and 3x3 box
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == digit || board[i][col] == digit ||
                board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] == digit) {
                return false;
            }
        }
        return true;
```

```
    }
}
```

Quiz Questions

1. What is the main characteristic that distinguishes backtracking from regular recursion? a) Backtracking is faster b) Backtracking explores multiple paths and undoes choices that don't work c) Backtracking only works with arrays d) Backtracking doesn't use recursion

Answer: b) Backtracking explores multiple paths and undoes choices that don't work

2. In the backtracking template, what happens after making a choice and recursively exploring? a) The function always returns true b) The choice is permanently made c) The choice is undone (backtracked) if the recursive call fails d) A new array is created

Answer: c) The choice is undone (backtracked) if the recursive call fails

3. Which of the following problems is best suited for backtracking? a) Finding the sum of array elements b) Sorting an array c) Finding all possible combinations of elements d) Computing factorial

Answer: c) Finding all possible combinations of elements

4. In the N-Queens problem, what does the backtracking algorithm do when it finds that placing a queen in a position leads to no solution? a) It stops the algorithm b) It removes the queen and tries the next position c) It prints an error message d) It places the queen anyway

Answer: b) It removes the queen and tries the next position

5. What is the base case in most backtracking algorithms? a) When an error occurs b) When the recursion depth exceeds a limit c) When a complete solution is found or all possibilities are exhausted d) When the array is empty

Answer: c) When a complete solution is found or all possibilities are exhausted

---

Practice Coding Questions

1. Word Search Given a 2D board and a word, find if the word exists in the grid. The word can be constructed from letters of sequentially adjacent cells, where "adjacent" cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

2. Combination Sum Given an array of distinct integers `candidates` and a target integer `target`, return a list of all unique combinations of `candidates` where the chosen numbers sum to `target`. You

may choose the same number from `candidates` an unlimited number of times.

3. Palindrome Partitioning Given a string `s`, partition `s` such that every substring of the partition is a palindrome. Return all possible palindrome partitioning of `s`.

4. Letter Combinations of a Phone Number Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order. (Like old phone keypads where 2=ABC, 3=DEF, etc.)

5. Generate Parentheses Given `n` pairs of parentheses, write a function to generate all combinations of well-formed parentheses.